WISEConv: A Worklist-Driven GPU Runtime for Masked Convolution over Dense Feature Grids

Anonymous Author(s)

Abstract

Masked convolution promises to reduce vision-model inference latency by computing only positions an upstream policy marks active, yet existing paths sacrifice either throughput or useful-compute ratio and fail to translate sparsity into proportional latency reduction. We present WISEConv, a masked-convolution runtime built on the observation that output coordinates form the interface between position selection and the regular reduction datapath in the dense tiled pipeline. WISEConv replaces the dense tiled pipeline's contiguous coordinate source with an active-output worklist, restricting computation to active positions while retaining dense tensor layouts and the regular per-position reduction. To keep worklist construction cheap and sustain throughput as the number of active outputs changes from frame to frame, WISEConv fuses mask propagation with tile-local compaction, reuses coordinate metadata across compatible operators, and adapts its compute decomposition without host synchronization. We evaluate WISEConv on four perception models across six GPU platforms: four discrete GPUs and two Jetson platforms. WISEConv achieves a 1.57× geometric-mean end-to-end speedup over the fastest evaluated competing path selected for each workload-platform pair, reducing full-model latency by 22.1–46.5%. On the Jetson platforms, it also reduces per-frame board energy by 28.5–72.6% relative to dense execution.

Keywords: Sparse convolution, dynamic spatial sparsity, GPU runtime, deep learning systems, efficient inference

1 Introduction

Dense convolution dominates the per-frame inference latency of many latency-critical vision tasks [3, 10, 39], yet on a given input much of that computation is spatially redundant. A common remedy is masked convolution: an upstream policy marks active positions, and the operator computes only those. The mask may derive from event-camera increments [8, 24], temporal differences [17, 29], or learned spatial gates [22, 36]. Regardless of source, it poses one operator-level problem: compute the convolution only at the active positions of the dense feature grid.

Dense GPU convolution obtains high throughput from tiled, coordinate-driven execution. Each tile enumerates a contiguous block of output coordinates; those coordinates fix each output position's receptive field and write location, while the reduction over channels and kernel offsets follows a regular datapath. Neighboring coordinates share overlapping receptive fields, yielding predictable memory access and data reuse. However, unlike static sparsity in matrix multiplication, masked convolution introduces runtime position selection per frame, requiring the operator to access scattered, irregularly-overlapping neighborhoods and handle a varying workload, conflicting with tiled execution.

Existing masked-convolution systems retain tiled execution or achieve precise position selection, but not both. *Tile skipping* [8, 17, 29, 31] retains the dense pipeline and suppresses a tile only when all its outputs are inactive, so a single active position triggers computation for the entire tile. *Gather-scatter* [22, 36, 40] (as an execution path) instead extracts active positions exactly, materializes receptive fields, and scatters the computation results to the dense grid, sacrificing regularity and incurring extraction, irregular access, and writeback overhead.

We characterize this trade-off along two axes: *throughput* (operations per second) and *useful-compute ratio* (the fraction of issued operations that produce needed outputs). In one of our workloads (FireFlowNet [27] under an event camera [8] mask policy, §2.2), the masks render 73.7% of the convolution work unnecessary, yet neither approach turns this into proportional latency reduction. Tile skipping [29] still wastes 26.4% of its issued operations and only cuts latency by at most 42.0%. Gather-scatter drives the useful-compute ratio near one but sustains only 21.7% of tile skipping's throughput, running 1.97× slower than the dense path.

Our key insight is that output coordinates form the interface between position selection and the reduction datapath in the dense tiled pipeline; replacing the contiguous coordinate source with an active-output worklist therefore preserves dense tensor layouts and the regular per-position reduction while computing only active outputs, opening a path to combine a high useful-compute ratio with the dense pipeline's throughput potential.

Realizing this potential requires solving two challenges. First, *worklist construction cost*: construction scans the mask, so its cost scales with the layer's spatial grid, not with the active count or channel widths, while the compute it enables shrinks with both. Construction's share of latency therefore grows as activity falls and channels thin, and accumulates across operators when every layer rebuilds. Second, *end-to-end worklist efficiency*: the worklist's order and length both affect its consumption throughput. An unordered worklist scatters the input neighborhoods that adjacent work touches, lowering locality; its length falls below the dense grid and shifts with layer and input, so no fixed tiling keeps the GPU full, and because the length is device-resident, any host-side decision that depends on it costs a synchronization on the per-frame path. Both effects sharpen as activity falls.

We present WISEConv (**W**orklist-dr**I**ven ma**S**k**E**d **Conv**olution), a masked-convolution runtime that fuses position-level selectivity into the dense tiled pipeline, securing both a high useful-compute ratio and high throughput by co-designing worklist construction and consumption. Construction is cheap and infrequent: a single mask-space pass fuses output-mask propagation with per-tile compaction, operating on only mask and coordinate metadata; compatible operators reuse valid metadata instead of reconstructing it. Consumption is dense-like: construction emits contiguous per-tile segments ordered by the dense tile traversal, giving the compute stage tile-local structure without global sorting. Compute adapts its parallel decomposition to runtime activity, selecting a per-operator configuration from an offline-calibrated mapping indexed by device, layer shape, and activity level; WISEConv leverages temporal correlation across successive inputs in streaming workloads, such as video and event streams, by using the latest completed, lagged input-activity observation to select each layer's configuration, sustaining GPU utilization without host synchronization.

This paper makes the following contributions:

- We characterize masked-convolution latency along two axes, useful-compute ratio and throughput, exposing why existing work does not translate upstream sparsity into proportional latency reduction.
- We design WISEConv, which fuses position-level selectivity into the dense tiled convolution pipeline and co-designs worklist construction and consumption to secure both axes jointly.
- We evaluate WISEConv on four perception models across six GPU platforms: four discrete GPUs and two Jetson platforms. WISEConv is the fastest path in all 24 workload-platform pairs, achieving a 1.57× geometric-mean speedup over the fastest evaluated competing path in each pair (22.1–46.5% lower full-model latency) and reducing per-frame board energy by 28.5–72.6% relative to dense execution on the Jetson platforms.

2 Background and Motivation

2.1 Masked Convolution

Convolution runs inside the perception-action loop of many vision applications [3, 33, 37, 39, 43], where per-frame inference latency bounds system responsiveness [9–11, 18]. To cut this latency, masked convolution exploits dynamic spatial sparsity: an upstream policy emits a spatial mask, and the operator computes only the marked positions. The mask comes from sources including event-camera increments [8, 24], temporal residuals between video frames [4, 17, 28, 29, 38], or learned input-dependent gates [13, 22, 36, 40]. The event and temporal policies mark positions that changed since the previous frame; the learned gates mark positions the task treats as relevant. These sources differ in sensing modality and policy, but all expose the same dynamic spatial sparsity on a dense 2D feature grid, so a single masked-convolution runtime serves them all.

Masked convolution keeps the tensors and arithmetic of dense convolution and restricts only which output coordinates it computes. It operates on an input feature tensor $\mathbf{x} \in \mathbb{R}^{H_{in} \times W_{in} \times C_{in}}$, a weight tensor $\mathbf{w} \in \mathbb{R}^{k \times k \times C_{in} \times C_{out}}$ of kernel size k, and an output $\mathbf{y} \in \mathbb{R}^{H_{out} \times W_{out} \times C_{out}}$ on a coordinate grid $\mathcal{G} = \{1, \ldots, H_{out} W_{out}\}$, where each coordinate $p \in \mathcal{G}$ indexes a spatial position (h, w) and has a $k \times k$ receptive field $\mathcal{N}(p)$ in $\mathbf{x}$. An upstream policy marks the coordinates to compute in an output mask $M_{out} \in \{0, 1\}^{H_{out} \times W_{out}}$. Spatial gating predicts M_{out} directly; event and temporal policies instead supply an input mask $M_{in} \in \{0, 1\}^{H_{in} \times W_{in}}$ whose active positions propagate their influence along the receptive field, so p stays active whenever any input in $\mathcal{N}(p)$ is active,

$$M_{out}[p] = \begin{cases} 1, & \exists\, r \in \mathcal{N}(p) \text{ s.t. } M_{in}[r] = 1, \\ 0, & \text{otherwise.} \end{cases} \tag{1}$$

M_{out} thus identifies the output positions that the execution path is required to compute, each by aggregating $\mathbf{x}$ over $\mathcal{N}(p)$ with the weights $\mathbf{w}$.

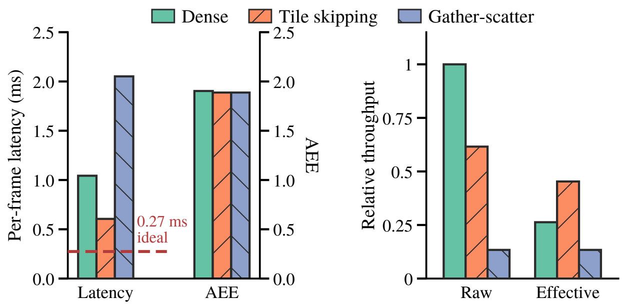
An execution path issues computation over a sequence of position slots C, where each slot is drawn from $\mathcal{G} \cup \{\perp\}$; $\perp$ denotes a padding slot that carries no output coordinate. The path must satisfy the *coverage contract*: every active coordinate is included, $\{p : M_{out}[p] = 1\} \subseteq \{c \in C : c \neq \perp\}$. For each non-$\perp$ slot $p \in C$ it computes

$$\mathbf{y}[p] = \sum_{i=1}^{k^2} \mathbf{x}[\mathcal{N}_i(p)]\, \mathbf{w}_i, \tag{2}$$

where $\mathcal{N}_i(p)$ is the i-th position in $\mathcal{N}(p)$ and $\mathbf{w}_i \in \mathbb{R}^{C_{in} \times C_{out}}$ the corresponding weight slice. How unrequested positions are represented or merged into a dense result is policy-specific. Event and temporal execution retains prior state, whereas learned gating merges the computed branch values into a residual or bypass tensor. Coverage is therefore the operator-level correctness condition: every position requested by the policy is computed, while the policy determines the treatment of all other positions. Beyond coverage, a path may issue inactive or $\perp$ slots; dense convolution is the extreme where $\{c \in C : c \neq \perp\} = \mathcal{G}$. The policy fixes the accuracy and required work, whereas the execution path determines the issued sequence C.

2.2 The Sparsity-to-Latency Gap

Existing systems execute a mask along one of two paths. *Tile skipping* [8, 17, 23, 28, 31, 38] inherits the dense pipeline's tile-based computation without additional metadata passes, skipping a tile only when all its coordinates are inactive and otherwise computing the whole tile; DeltaCNN [29]



(a) Latency and accuracy (b) Relative throughput

Figure 1. FireFlowNet [27] using the Ev-Conv [8] event-driven mask policy on MVSEC [44] and RTX 3080, comparing cuDNN (Dense), DeltaCNN tile skipping [29], and MMCV's native gather-scatter path [5]. (a) Mean full-model per-frame latency and average endpoint error (AEE; lower is better). The dashed line marks dense latency scaled by the active ratio aggregated across layers; the sparse paths use the same masks and produce nearly identical AEE. (b) Raw throughput T and effective throughput T_{eff}, each normalized to dense raw throughput. The dense useful-compute ratio equals the active ratio set by the Ev-Conv mask policy ($\alpha \approx 26.3\%$).

fuses a selective path into the kernel, taken when a tile is estimated to be mostly empty, to cut wasted work. *Gather-scatter* [22, 40] instead abandons the dense pipeline: it extracts exactly the active coordinates into a packed representation, runs a separate GEMM, and scatters the results back, removing the inactive arithmetic but adding extraction, irregular access, and writeback overhead; DynConv [36] adopts a model structure suited to gather-scatter, where one gather feeds several consecutive layers before a single scatter, amortizing that overhead.

These paths fail for different reasons, so we separate policy sparsity from execution efficiency. The policy fixes its active ratio α, the fraction of output-grid positions marked active. An execution path instead determines its useful-compute ratio η and throughput T, which combine into effective throughput T_{eff}:

$$\begin{aligned} \alpha &= \frac{\|M_{out}\|_0}{|\mathcal{G}|}, &\qquad \eta &= \frac{\|M_{out}\|_0}{|C|} \le 1, \\ T &= \frac{2|C|k^2 C_{in} C_{out}}{t}, &\quad T_{\text{eff}} &= \eta T. \end{aligned} \tag{3}$$

For $|C| > 0$, η captures inactive and padding slots in the issued schedule, while T captures how quickly the path processes those slots. The latency t includes all auxiliary mask and coordinate work, so such work lowers T without changing η; the factor of two counts each multiply-accumulate as two FLOPs. Consequently, T_{eff} measures the rate of required-output computation and therefore determines its latency.

Figure 1 traces this on RTX 3080 for FireFlowNet [27] under the Ev-Conv [8] mask policy. Event cameras expose microsecond-scale changes, and Ev-Conv evaluates optical flow on event-window inputs updated at 1 kHz. The policy marks 73.7% of the convolution work unnecessary, and the sparse paths discard it without reducing accuracy. Scaling dense latency by the fraction of convolution work that remains gives a 0.274 ms ideal proportional-scaling reference (Fig. 1a, dashed). Neither path approaches it: tile skipping (DeltaCNN [29]) reaches 0.606 ms, still 2.21× the reference, and gather-scatter (MMCV [5]) runs at 2.052 ms, almost twice the latency of dense convolution itself. The gap traces to low effective throughput on both paths. Tile skipping holds throughput at 61.6% of dense but at a useful-compute ratio of only 73.6%; gather-scatter lifts the useful-compute ratio to one, yet extraction and data movement collapse its throughput to 13.4% of dense (Fig. 1b). Neither path efficiently converts the GPU's capability into effective throughput.

2.3 Distinction from Related Sparse Execution

Other related sparse-execution work differs from WISEConv in representation, operator, sparsity type, or platform. 3D sparse-convolution systems [7, 16, 32, 34, 35, 41] operate on sparse point-cloud coordinates and construct coordinate maps for irregular input-to-output relations; WISEConv instead consumes a runtime mask over a dense 2D grid while retaining dense tensor layouts. Sparse matrix multiplication [12, 14, 30] accelerates sparse-graph and pruned-weight matrix products, where sparsity is structural rather than a dynamic spatial mask over a convolution grid. Structured weight pruning (e.g., 2:4 sparsity [26]) exploits fixed patterns in model weights. Sparse-format compilers [21, 42] expose sparsity through explicit tensor formats, whereas WISEConv retains dense layouts and uses a transient per-frame mask only to select output coordinates. DPACS [15] resolves similar dynamic spatial sparsity in CNN feature maps, but on a custom accelerator instead of a commodity GPU.

Some 3D systems [32, 35] compute through coordinate-indexed fused GEMM [1], where an index array derived from coordinate maps redirects the position dimension of a tiled GEMM. It seemingly coincides with WISEConv's goal, but 3D sparse convolution requires explicit coordinate metadata because its tensors are represented as sparse coordinate sets. Constructing the index array incurs hashing, sorting, and global scans that frequently dominate per-layer latency over the GEMM itself [35, 41], making construction cost one of the central research problems in the 3D community; thus directly introducing coordinate indirection into 2D convolution which already tiles densely appears unattractive. 2D masked convolution, however, presents a structurally simpler mapping problem: the output grid is regular, the mask is a bitmap, and each coordinate's receptive-field addresses follow from fixed grid arithmetic, so the coordinate-map machinery of 3D systems is unnecessary. For 2D masked convolution, WISEConv identifies the coordinate source as the replaceable interface between position selection and tiled reduction, while the co-design of worklist construction and consumption realizes this insight to secure both a high useful-compute ratio and high throughput.

3 WISEConv Overview

Dense tiled convolution already consumes a stream of output coordinates: each coordinate determines the receptive-field reads and output write, while its reduction over channels and kernel offsets remains regular. WISEConv exploits this interface by replacing the dense coordinate sequence with a worklist of requested output positions, thereby introducing position-level selectivity into the tiled pipeline. Removing unrequested positions from the schedule raises the useful-compute ratio, but issuing fewer operations alone does not guarantee lower latency. To translate selectivity into latency reduction, the worklist must be inexpensive to construct and must be consumed with both a high useful-compute ratio and high throughput. WISEConv therefore co-designs a construction stage and a compute stage around these interdependent requirements, as Figure 2 summarizes.

Construction stage. To prevent worklist construction from erasing the savings of selective compute, WISEConv confines construction to mask and coordinate metadata, never reading feature or weight tensors. In one mask-space pass, it fuses the output-mask propagation in Eq. 1 with tile-local compaction, producing M_{out} and a worklist $\mathcal{W}$; the requested coordinates from each spatial tile form a segment in local dense-traversal order, preserving spatial grouping without global ordering. WISEConv stores $\mathcal{W}$ in a buffer preallocated for $|\mathcal{G}| = H_{out}W_{out}$ coordinates, the static upper bound on $|\mathcal{W}|$. A device-resident counter records $n_{\mathcal{W}} = |\mathcal{W}|$, so construction needs neither runtime allocation nor a host-visible active count.

Even this metadata-only pass must scan the layer's mask grid. WISEConv therefore propagates and reuses the resulting mask and worklist metadata across compatible operators instead of reconstructing it, amortizing one construction across the compatible operator sequence. The fused pass lowers the cost of each construction, while cross-operator reuse reduces how often it occurs; together, they address the Worklist Construction Cost challenge.

Compute stage. The worklist length varies across layers and inputs and remains device-resident; synchronizing its current value to the host would stall the per-frame path, yet the workload decomposition must adapt because its padded work and exposed parallelism govern useful-compute ratio and GPU throughput. To execute without a host-visible bound, WISEConv launches a persistent grid whose size is capped by the smaller of the GPU's concurrent block residency and the layer configuration's dense work bound, then lets the blocks consume $\mathcal{W}$ through a grid-stride loop bounded by device-resident $n_{\mathcal{W}}$.

To choose a decomposition without querying the exact $n_{\mathcal{W}}$, WISEConv builds from representative temporal traces a table that maps input-activity levels to configurations that sustain throughput while limiting padding, then indexes it with the most recent completed observation from one or more frames earlier; temporal correlation keeps this lagged signal informative, while completed-only polling prevents selection from blocking. The device count thus bounds the current work, while the data-driven signal chooses its decomposition; together, they sustain useful-compute ratio and GPU throughput as the worklist length varies without synchronizing the per-frame path.

Neither stage suffices alone: excessive construction cost, a low useful-compute ratio, or low consumption throughput can each erase the benefit of issuing less convolution work. By controlling all three jointly, WISEConv converts upstream spatial sparsity into end-to-end latency reduction.

4 WISEConv Design

The design follows the worklist from its construction and propagation across compatible operators (§4.1) to its consumption by the convolution pipeline (§4.2).

4.1 Worklist Construction and Reuse

4.1.1 Tile-Local Worklist Construction. Constructing a tight schedule already requires a scan of the output grid, but the resulting coordinates must also retain enough spatial structure for efficient consumption. Globally stable compaction preserves the complete dense traversal order but requires a prefix scan or another pass to obtain device-wide offsets; per-coordinate appends avoid that coordination but scatter neighboring positions. Materializing active receptive fields, as gather-scatter does, regularizes the compute input at the cost of feature traffic proportional to the active count and k^2C_{in}. The useful middle ground is therefore a metadata-only schedule with local rather than global ordering.

WISEConv realizes this middle ground with contiguous, tile-local worklist segments. Because the number of active positions is not known before the pass, it preallocates a coordinate buffer with capacity $|\mathcal{G}| = H_{out}W_{out}$, the static upper bound on $|\mathcal{W}|$. Algorithm 1 details the resulting single pass. Line 1 initializes the device-resident length counter. Lines 2–7 partition the output grid into $B_\tau \times B_\tau$ construction tiles, derive the corresponding entries of M_{out} using Eq. 1, and compact each tile's active coordinates into a local sequence $\mathcal{P}_\tau$ in dense-traversal order. When a learned gate supplies M_{out} directly [22, 36], Lines 4–5 are omitted and Line 7 compacts the supplied mask. Lines 8–11 count the compacted coordinates, atomically reserve $[\text{base}, \text{base} + n_\tau)$ from the counter, and copy $\mathcal{P}_\tau$ into that interval. After all tiles complete, the reserved intervals collectively store $\mathcal{W}$, and the counter records $n_{\mathcal{W}}$. Because the layer shape fixes the buffer capacity and launch geometry while only the device observes

[TODO] WISEConv overview. Left: the mask-space construction stage propagates the output mask, emits tile-local worklist segments into a preallocated buffer, and records the worklist length on the device; compatible operators reuse this metadata. Right: the device-bounded worklist is consumed by a persistent grid capped by concurrent block residency and the layer configuration's dense work bound, while the most recent completed input-activity observation from an earlier frame selects a high-throughput, low-padding workload decomposition.

Figure 2. The WISEConv two-stage pipeline. Construction emits tile-locally ordered worklist segments using only mask and coordinate metadata, then reuses the metadata across compatible operators. Compute launches a grid capped by the layer configuration's dense work bound and concurrent block residency, then adapts its decomposition from a completed, temporally lagged input-activity observation without host synchronization.

the data-dependent length, construction requires no runtime allocation or host-visible active count.

To relate the worklist to the useful-compute ratio of Eq. 3, we insert its length $n_{\mathcal{W}}$ between the required-position count and the issued-slot count. For $n_{\mathcal{W}} > 0$,

$$\eta = \underbrace{\frac{\|M_{out}\|_0}{n_{\mathcal{W}}}}_{\eta_{\text{construct}}} \cdot \underbrace{\frac{n_{\mathcal{W}}}{|C|}}_{\eta_{\text{compute}}} . \tag{4}$$

Here, $\eta_{\text{construct}}$ compares required positions with worklist entries, while η_{compute} compares worklist entries with issued slots. We analyze the former here and the latter after defining the compute organization in §4.2. The factorization decomposes η only; all stage costs remain reflected in throughput T.

Because construction tiles partition the output grid and use disjoint reservations, each active coordinate appears exactly once, so $n_{\mathcal{W}} = \|M_{out}\|_0$. Fresh construction therefore has $\eta_{\text{construct}} = 1$ for a nonempty worklist. Coverage constrains membership rather than order, so tile reservation order does not affect correctness; traversal order within each interval is retained for locality.

WISEConv thus confines its ordering guarantee to each contiguous segment: coordinates retain the tile's dense traversal, while independently reserved segments provide no cross-boundary locality guarantee. A B_M-wide compute tile may therefore span coordinates from multiple construction segments when segments are shorter than B_M; the preserved locality is intra-segment contiguity, not compute-tile-to-construction-tile alignment. As shown in §6.4, this tile-local order achieves compute-stage throughput comparable to global ordering, making the additional ordering pass an unfavorable end-to-end trade-off.

Algorithm 1 Tile-Local Worklist Construction

Require: Input mask M_{in}, output grid $\mathcal{G}$, tile size B_τ
Ensure: Mask M_{out}, worklist $\mathcal{W}$, length $n_{\mathcal{W}} = |\mathcal{W}|$

```
n_W ← 0
for each B_τ × B_τ spatial tile τ in parallel do
    for each position p ∈ τ in parallel do
        active(p) ← ∃ r ∈ N(p) : M_in[r] = 1
        M_out[p] ← active(p)
    end for
    P_τ ← COMPACT(τ, M_out)
    n_τ ← |P_τ|
    base ← AtomicAdd(n_W, n_τ)
    for each 0 ≤ j < n_τ in parallel do
        W[base + j] ← P_τ[j]
    end for
end for
```

4.1.2 Cross-Operator Worklist Reuse. Each construction pass examines all $H_{out}W_{out}$ positions and, when deriving M_{out} from M_{in}, performs $O(k^2)$ mask work per position. This channel-independent grid scan contrasts with required convolution work, which scales as $O(\alpha H_{out} W_{out} k^2 C_{in} C_{out})$. Repeating the scan across same-grid layers that introduce no new required positions is therefore redundant. WISEConv instead amortizes construction over each compatible layer sequence by reusing one worklist across its operators.

For layer l, let $\mathcal{G}^l$ denote its output grid, M^l_{out} its output mask, $\mathcal{W}^l$ its worklist, and $n^l_{\mathcal{W}}$ its device-resident length. WISEConv propagates the mask and worklist metadata alongside the feature tensor. From layer semantics, WISEConv statically infers that consecutive layers are reuse-compatible when $\mathcal{G}^{l+1} = \mathcal{G}^l$ and

$$\{p \in \mathcal{G}^{l+1} \mid M^{l+1}_{out}[p] = 1\} \subseteq \{p \in \mathcal{G}^l \mid M^l_{out}[p] = 1\}. \quad (5)$$

Because $\mathcal{W}^l$ covers M^l_{out}, Eq. 5 guarantees coverage for M^{l+1}_{out}. WISEConv therefore forwards $\mathcal{W}^{l+1} = \mathcal{W}^l$ and $n^{l+1}_{\mathcal{W}} = n^l_{\mathcal{W}}$ without reconstruction and applies the condition recursively at each subsequent layer.

Equality in Eq. 5 covers same-grid 1×1 convolutions, mask-preserving activations, and inference-time normalization, leaving $\eta_{\text{construct}}$ unchanged. Strict inclusion can result from mask-narrowing operators in the upstream policy, such as the activation-fused update truncation that DeltaCNN uses to increase downstream sparsity [29]. The inherited worklist remains a valid superset, trading a lower $\eta_{\text{construct}}$ for one fewer construction pass without violating coverage. Because compute gates all output writes by the exact mask, positions scheduled but no longer active under the current mask participate in reduction yet produce no store; the superset schedule therefore cannot alter any inactive position's observable state.

A change in the output grid violates the geometry constraint, while receptive-field expansion may violate Eq. 5 by introducing positions outside the upstream mask; either case invokes the construction pass of §4.1.1 to emit a new tight worklist. A lower $\eta_{\text{construct}}$ alone does not trigger reconstruction because tightening a valid superset would pay another grid scan. Common receptive-field-expanding layers such as 3×3 convolutions still create rebuild points throughout the network, leaving device-resident worklist lengths variable across layers and frames. Compute must therefore handle a dynamic problem size.

4.2 Variable-Length Worklist Execution

Construction and reuse fix $\eta_{\text{construct}}$; the implicit-GEMM decomposition controls η_{compute} and throughput T. For a standard ungrouped batch-1 convolution, dense implicit GEMM uses $M = H_{out}W_{out}$ output-position rows, $N = C_{out}$ output-channel columns, and reduction depth $K = k^2 C_{in}$, with $B_M \times B_N$ output tiles and B_K-wide reduction steps. WISEConv preserves the N and K dimensions and this tiled pipeline, replacing only the M-dimension coordinate source with $\mathcal{W}$. Compute therefore groups $\mathcal{W}$ into B_M-coordinate tiles, padding only the final tile. For $n_{\mathcal{W}} > 0$, this issues $|C| = B_M \lceil n_{\mathcal{W}} / B_M \rceil$ position slots, giving

$$\eta_{\text{compute}} = \frac{n_{\mathcal{W}}}{B_M \lceil n_{\mathcal{W}} / B_M \rceil}. \quad (6)$$

The final tile contains at most $B_M - 1$ padding slots, making this overhead most pronounced for a short worklist. The same B_M also controls data reuse, block efficiency, and exposed parallelism, so maximizing η_{compute} need not maximize throughput T.

Because dense (M, N, K) follow from the layer shape, the backend can select (B_M, B_N, B_K) before execution. In the worklist path, however, the M-dimension extent is $n_{\mathcal{W}}$, which varies across invocations and remains unknown to the host. This leaves two problems: executing the current worklist without exposing its length to the host, and selecting a decomposition suited to that length. §4.2.1 addresses the former with persistent execution, while §4.2.2 addresses the latter through trace-calibrated selection.

4.2.1 Persistent Worklist-Driven Convolution.

Dense row m obtains its coordinate p_m from the output-grid traversal; WISEConv instead sets $p_m = \mathcal{W}[m]$. Because p_m determines the receptive-field and output addresses, only the input gathers and output stores change. Feature and weight layouts and the regular B_K-tiled reduction remain unchanged, and no receptive-field patches are materialized. Contiguous ranges of $\mathcal{W}$ carry construction's tile-local grouping into the gathers.

Because the host chooses a conventional launch's thread-block count, using device-resident $n_{\mathcal{W}}$ to size the grid would require a synchronizing transfer. For the $S_K = 1$ path in Algorithm 2, the dense output grid defines the static upper bound

$$n^{\max}_{\text{work}} = \left\lceil \frac{|\mathcal{G}|}{B_M} \right\rceil \left\lceil \frac{C_{out}}{B_N} \right\rceil. \quad (7)$$

Let g_{res} denote the concurrent block capacity determined by the GPU's occupancy and SM count. WISEConv launches

$$g_{\text{launch}} = \min\left(g_{\text{res}}, n^{\max}_{\text{work}}\right) \quad (8)$$

blocks. This launch bound depends on the layer shape and configuration but not on the current $n_{\mathcal{W}}$. Each block reads this device-resident length and consumes valid work through a grid-stride loop. If the dense upper bound is smaller than the residency capacity, the grid uses that bound directly. The current worklist length remains device-resident, so no host synchronization is needed.

Algorithm 2 assigns work entirely on the device. Line 1 derives the number of position-channel work items from $n_{\mathcal{W}}$; Lines 2–3 distribute them across g_{launch} persistent blocks in a grid-stride loop. For each item, Lines 4–8 recover its (b_m, b_n) indices, load and pad the corresponding B_M-coordinate slice, select its B_N output channels, and initialize a fixed $B_M \times B_N$ accumulator. Lines 9–16 perform the regular implicit-GEMM reduction: each kernel offset and B_K-wide channel slice forms $\mathbf{X}$ and $\mathbf{W}$ directly from the dense tensors (Lines 12–13) and accumulates $\mathbf{XW}$ (Line 14). Line 17 writes only rows whose coordinates are active in the current M_{out}, after which

Algorithm 2 Persistent Worklist-Driven Implicit GEMM

Require: Device-resident $(M_{out}, \mathcal{W}, n_{\mathcal{W}})$, persistent-grid launch bound g_{launch}, tensors $(\mathbf{x}, \mathbf{w})$, and tile shape (B_M, B_N, B_K)

Ensure: Output $\mathbf{y}$ (updated at worklist coordinates)

1: $n_{\text{work}} \leftarrow \lceil n_{\mathcal{W}}/B_M \rceil \lceil C_{out}/B_N \rceil$
2: **for** $t_i = 0, \ldots, g_{\text{launch}} - 1$ **in parallel do**
3:   **while** $t_i < n_{\text{work}}$ **do**
4:     $(b_m, b_n) \leftarrow \text{Unflatten}(t_i)$
5:     $\mathcal{P} \leftarrow \mathcal{W}[b_m B_M : \min((b_m+1)B_M, n_{\mathcal{W}})]$
6:     Pad $\mathcal{P}$ to B_M entries with $\perp$
7:     $c_1 \leftarrow b_n B_N, \quad c_2 \leftarrow (b_n+1)B_N$
8:     $\mathbf{Y} \leftarrow \mathbf{0}_{B_M \times B_N}$
9:     **for** $j = 1, \ldots, k^2$ **do**
10:       **for** $r = 0, \ldots, \lceil C_{in}/B_K \rceil - 1$ **do**
11:         $r_1 \leftarrow r B_K, \quad r_2 \leftarrow (r+1)B_K$
12:         $\mathbf{X} \leftarrow [\, \mathbf{x}[\mathcal{N}_j(p), r_1 : r_2] \,]_{p \in \mathcal{P}}$, zero-padding $\perp$
13:         $\mathbf{W} \leftarrow \mathbf{w}_j[r_1 : r_2, c_1 : c_2]$
14:         $\mathbf{Y} \leftarrow \mathbf{Y} + \mathbf{X} \cdot \mathbf{W}$
15:       **end for**
16:     **end for**
17:     $\mathbf{y}[\mathcal{P}, c_1 : c_2] \leftarrow \mathbf{Y}$ for $p \in \mathcal{P}$ with $M_{out}[p] = 1$, skipping $\perp$
18:     $t_i \leftarrow t_i + g_{\text{launch}}$
19:   **end while**
20: **end for**

Line 18 advances the block by g_{launch}. For clarity, the pseudocode omits the boundary checks required when C_{in} or C_{out} is not tile-aligned. Thus, $n_{\mathcal{W}}$ controls the device-side work bound and final position padding, while M_{out} gates observable stores; the multiply shape and tensor layouts remain fixed, leaving only the choice of an efficient decomposition for this bound.

4.2.2 Trace-Calibrated Adaptive Decomposition. Persistent scheduling resolves the execution bound, but not the decomposition choice: for a fixed layer, the fastest configuration changes with $n_{\mathcal{W}}$. A configuration $q = (B_M, B_N, B_K, S_K)$ includes a split-K factor S_K, the number of disjoint partitions of the $K = k^2 C_{in}$ reduction for each position-channel tile. Algorithm 2 presents the $S_K = 1$ path, which writes each completed reduction directly. When $S_K > 1$, each position-channel tile is replicated across S_K disjoint reduction partitions, multiplying the dense work bound in Eq. 7 by S_K; Eq. 8 then applies to this expanded bound. Before the split-K kernels run, a mask-gated zeroing kernel executes on the same CUDA stream, after which the partitions atomically accumulate their partial sums. Both zeroing and accumulation use the current exact output mask. For layer l, the values supported on the target GPU define the configuration domain

$$\mathcal{Q}_l = \mathcal{B}_M^l \times \mathcal{B}_N^l \times \mathcal{B}_K^l \times \mathcal{S}_K^l,$$

where each factor contains the supported values of its corresponding parameter.

The length-sensitive effect is clearest in B_M, which trades final-tile padding and exposed position parallelism against within-tile reuse. The same choice also controls the work exposed to each persistent block, so maximizing η_{compute} need not maximize throughput T.

WISEConv builds a level-to-configuration table $\mathcal{L}$, with one row $\mathcal{L}_l$ per layer, after quantizing α into D levels as $d = \max\{1, \lceil D\alpha \rceil\}$. Calibration deliberately represents each level with a fresh, tight worklist rather than reproducing every reuse state. For each layer l and level d, WISEConv constructs a synthetic worklist containing $\lceil d|\mathcal{G}^l|/D \rceil$ active coordinates with construction's tile-local structure, then measures $\widehat{t}_l(q; d)$, the latency of every $q \in \mathcal{Q}_l$ at that level. It records

$$q^*_{l,d} = \arg\min_{q \in \mathcal{Q}_l} \widehat{t}_l(q;\, d). \tag{9}$$

Because all worklist candidates execute the same synthetic worklist, Eq. 9 selects the minimum-latency decomposition, equivalently the one with the highest effective throughput at that level. WISEConv stores it as $\mathcal{L}_l[d] = q^*_{l,d}$. When full-grid execution is semantics-compatible, calibration also compares a dense candidate as a special case and stores it in the same table if it is faster.

Lookup is cheap once d is known, but obtaining every layer's current level directly would require transferring its device-resident activity count to the host. Waiting would synchronize each layer. Making these transfers asynchronous and using the latest completed values removes the waits, but the runtime must still enqueue and track one scalar transfer per layer per frame, so observation overhead scales with the layer count. WISEConv deliberately rejects both alternatives: it enqueues only one asynchronous transfer of network-input activity per frame, then uses the latest completed, necessarily lagged observation to predict every layer's level. This design leverages temporal correlation across successive inputs in streaming workloads whose layer masks derive from a shared network-input mask: the latest completed, necessarily lagged observation then predicts every layer's level.

To fit this predictor, WISEConv profiles a set $\mathcal{F}$ of representative trace frames for the set $\mathcal{I}$ of layers that use it, recording $\{\alpha_0^f\}_{f \in \mathcal{F}}$ and $\{\alpha_l^f\}_{(f,l) \in \mathcal{F} \times \mathcal{I}}$. It partitions the observed input-activity range $[\alpha_{\text{lo}}, \alpha_{\text{hi}}]$ into B equal-width buckets, with $\alpha_{\text{lo}} = \min_{f \in \mathcal{F}} \alpha_0^f$ and $\alpha_{\text{hi}} = \max_{f \in \mathcal{F}} \alpha_0^f$. Frame f receives index $b^f \in \{1, \ldots, B\}$:

$$b^f = \max\left\{1, \left\lceil \frac{B(\alpha_0^f - \alpha_{\text{lo}})}{\alpha_{\text{hi}} - \alpha_{\text{lo}}} \right\rceil \right\}. \tag{10}$$

Equal-width buckets assign fixed activity ranges independent of sample frequency and therefore preserve resolution in rare regimes.

Given these frame groups, WISEConv fits a bucket-to-level prediction table $\mathcal{A}$ with layer-specific rows $\mathcal{A}_l$, since grid resolution, receptive-field size, and propagation depth transform the same input activity differently at each layer. For every bucket b, WISEConv computes the median layer activity $\widetilde{\alpha}_{l,b} = \text{median}_{f\in\mathcal{F}:b^f=b}\,\alpha_l^f$ over the frames assigned to b, then quantizes it into the predicted level for layer l in bucket b:

$$\hat{d}_{l,b} = \max\left\{1, \left\lceil D\widetilde{\alpha}_{l,b}\right\rceil\right\}. \tag{11}$$

The median minimizes absolute deviation within the bucket and is robust to outliers. WISEConv stores $\mathcal{A}_l[b] = \hat{d}_{l,b}$ and fills empty buckets from their nearest populated neighbors.

At runtime, WISEConv composes $\mathcal{A}_l$ and $\mathcal{L}_l$ to translate the latest completed network-input activity observation into configurations for all layers. At frame f, if this observation comes from $f' < f$, WISEConv clamps $\alpha_0^{f'}$ to $[\alpha_{\text{lo}}, \alpha_{\text{hi}}]$ and applies Eq. 10 to obtain $\hat{b}^f$, avoiding extrapolation beyond trace coverage; if none has completed, it retains $\hat{b}^{f-1}$. It then selects

$$q_l^f = \mathcal{L}_l\left[\mathcal{A}_l[\hat{b}^f]\right]. \tag{12}$$

The observation may lag the current input, and reuse may make the runtime worklist longer than the predicted activity level implies. Both effects influence only the configuration: persistent compute still reads the current $n^l_{\mathcal{W}}$ on the device as the exact execution bound. Selection error can therefore affect performance but cannot omit scheduled coordinates.

Algorithm 3 connects the offline tables to nonblocking selection over the inference sequence $\mathcal{R}$. Lines 1–6 build $\mathcal{L}$ and $\mathcal{A}$ from the calibration and trace data; Lines 7–9 initialize the asynchronous queue and initial bucket. On each frame, Lines 10–17 retain the previous bucket if no observation has completed; otherwise, they clamp and bucket the latest value and compose $\mathcal{A}_l$ with $\mathcal{L}_l$ to select all layer configurations. Lines 18–21 derive and enqueue the current input active ratio asynchronously, and Line 22 executes the frame with the selected configurations. A value becomes visible only after its device-to-host transfer completes. The bounded queue returns the newest visible value and discards older ones; when no slot is free, enqueue drops the new observation. Neither operation waits, so selection never stalls construction or compute.

Learned-gating models do not derive layer masks from the network-input mask: each gated block produces its own mask from intermediate features, so a single input observation cannot predict per-layer activity. Observing their activity levels asynchronously would require one transfer per gated layer per frame, so WISEConv instead assigns each layer a fixed level calibrated from its gate statistics. The trace-driven mode thus uses one nonblocking input observation per frame, while the fixed-level mode uses none; neither adds per-layer device-to-host transfers to frame execution.

Algorithm 3 Trace-Calibrated Nonblocking Configuration Selection

Require: Calibration frames $\mathcal{F}$, inference sequence $\mathcal{R}$, layers $\mathcal{I}$, D, and B
Ensure: Inference results for $\mathcal{R}$
1: **Offline:**
2: $\mathcal{L} \leftarrow$ level-to-configuration table from Eq. 9
3: $\{\alpha_0^f\}_f, \{\alpha_l^f\}_{f,l} \leftarrow$ input and layer active-ratio traces
4: $(\alpha_{\text{lo}}, \alpha_{\text{hi}}) \leftarrow (\min_{f\in\mathcal{F}}\alpha_0^f, \max_{f\in\mathcal{F}}\alpha_0^f)$
5: $\{b^f\}_f \leftarrow$ input buckets of $\{\alpha_0^f\}_f$ by Eq. 10
6: $\mathcal{A} \leftarrow$ bucket-to-level table from Eq. 11
7: **Online:**
8: *Queue* $\leftarrow$ empty asynchronous queue
9: $\hat{b}^0 \leftarrow$ calibrated initial input bucket
10: **for** inference frame $f = 1, \ldots, |\mathcal{R}|$ **do**
11: $\quad\hat{b}^f \leftarrow \hat{b}^{f-1}$
12: $\quad$**if** *Queue* is not empty **then**
13: $\quad\quad\alpha_0^{f'} \leftarrow$ dequeueLatest(*Queue*) ▷ $f' < f$
14: $\quad\quad\alpha_0^{f'} \leftarrow \text{clamp}(\alpha_0^{f'}, \alpha_{\text{lo}}, \alpha_{\text{hi}})$
15: $\quad\quad\hat{b}^f \leftarrow$ input bucket of $\alpha_0^{f'}$ by Eq. 10
16: $\quad$**end if**
17: $\quad\{q_l^f\}_l \leftarrow \{\mathcal{L}_l[\mathcal{A}_l[\hat{b}^f]]\}_l$
18: $\quad$**async do**
19: $\quad\quad\alpha_0^f \leftarrow$ current input-mask active ratio (Eq. 3)
20: $\quad\quad$Enqueue α_0^f into *Queue*
21: $\quad$**end async**
22: $\quad$Run frame f with $\{q_l^f\}_l$ via Algorithms 1 and 2
23: **end for**

5 Implementation

We implement WISEConv as a PyTorch extension with a Python frontend and a CUDA backend written in C++. A model-conversion pass replaces supported modules, preserves their parameters and weights, and statically marks worklist-reuse edges and rebuild points. The compute backend adapts CUTLASS [1] dense tiled implicit-GEMM pipelines for direct worklist consumption, covering both FP32 (SIMT) and FP16 (tensor core) arithmetic with NHWC tensors. The full-workload evaluation uses the FP32 datapath; the DynConv sensitivity study additionally exercises the FP16 datapath. The backend supports standard, grouped, depthwise, and transposed convolutions, as well as the mask-aware auxiliary operators required by evaluated workloads. Each output carries a lightweight record that references its device-resident M_{out}, $\mathcal{W}$, and $n_{\mathcal{W}}$ buffers. All buffers are allocated during module initialization; mask selection remains with the upstream application.

The construction kernel maps one CUDA block to each construction tile in Algorithm 1. Warp ballots compute lane-local ranks, a short shared-memory prefix combines the warp counts, and one `atomicAdd` reserves the tile's contiguous worklist segment. Worklists use 32-bit linear coordinates and are provisioned for $|\mathcal{G}|$ entries; the runtime resets $n_{\mathcal{W}}$ on the operator's CUDA stream before each construction. When a learned gate supplies M_{out} directly, construction skips mask propagation and compacts active coordinates from the supplied mask. Compatible operators reuse the worklist and its length by aliasing the $\mathcal{W}$ and $n_{\mathcal{W}}$ buffers, while M_{out} always denotes the current operator's exact output mask.

The compute backend instantiates a C++ template family over (B_M, B_N, B_K, S_K) and sets each variant's persistent grid size from CUDA occupancy, the GPU's SM count, and the layer's dense work bound. The online selector (Algorithm 3) uses a bounded queue of pinned host scalars, CUDA events, and a dedicated copy stream. For event and temporal workloads, it transfers one network-input count per frame and selects each layer's configuration from the newest completed observation. Learned-gating models do not derive layer masks from the network-input mask, and the evaluated pose samples are not temporally correlated (§6.3); WISEConv therefore assigns fixed per-layer levels calibrated from gate statistics. Each invocation launches the fixed persistent grid of its selected configuration; the kernel reads device-resident $n_{\mathcal{W}}$ to bound its grid-stride loop. Every backend variant agrees numerically with cuDNN at each active output position.

6 Evaluation

6.1 Experimental Setup

Platforms. We evaluate six GPU platforms. The four discrete GPUs are NVIDIA A100, RTX 4090, RTX 3080, and RTX 4070 Laptop. The embedded platforms are Jetson Xavier NX and AGX Orin; we also measure board energy on both. The discrete platforms use PyTorch 2.1.2, CUDA 12.1, and cuDNN 8.9.2; both Jetson platforms use PyTorch 2.1.0, CUDA 11.4, and cuDNN 8.6. All paths in the full workload comparison use FP32 (SIMT) with TF32 disabled; the targeted DynConv sensitivity study also evaluates FP16 (tensor core) on RTX 3080. On each platform, all backends run under the same fixed clock and power configuration.

Models, tasks, and mask sources. Our workloads span three vision tasks.

- **Event-based optical flow.** FireFlowNet [27] is a lightweight event-based optical-flow model that delivers high inference rates while retaining competitive accuracy. We evaluate it on MVSEC [44], which contains event-camera sequences and corresponding ground-truth flow from indoor quadrotor flight and outdoor driving. Following Ev-Conv [8], consecutive 50-ms windows advance by 1 ms; events entering or leaving the window form the sparse increment from which we derive layer masks.
- **Detection.** We evaluate YOLOv8n and YOLOv8m [19] on MOT16 [25], a collection of crowded pedestrian videos for multi-object tracking. Their two model sizes expose how scale affects sparse-execution efficiency. For both models, DeltaCNN's temporal policy [29] thresholds activation differences between successive frames and propagates the resulting masks across layers.
- **Human pose estimation.** We use the DynConv pose model [36] on MPII [2], which depicts people performing diverse activities and annotates their 2D body joints. DynConv's learned spatial gates produce an input-dependent mask for each gated block from intermediate features, and MPII samples are temporally unrelated. We deliberately include this workload, where trace-driven adaptation is entirely disabled, to isolate and justify the worklist-driven execution model's latency gains independent of runtime adaptation.

Table 1 summarizes the workloads and evaluation scales. For every workload, calibration uses the training split, while all reported results use a disjoint test split. We use the authors' released weights for all four models [19, 27, 36]. Within each workload, we fix the inputs, weights, and upstream mask policy; every sparse path receives the same resulting layer masks, isolating the masked-convolution backend.

Table 1. Evaluation workloads.

	FireFlowNet [27]	**YOLOv8n/m** [19]	**DynConv** [36]
Task	Optical flow	Detection	Human pose
Mask source	Ev-Conv [8]	DeltaCNN [29]	DynConv [36]
Dataset	MVSEC [44]	MOT16 [25]	MPII [2]
Input size	256×256	640×640	256×256
Parameters	0.056M	3.16M / 25.9M	6.89M
Eval. samples	196.8K	2,575	2,958

Baselines. We compare WISEConv against three FP32 (SIMT) execution paths established by prior work:

- **Dense** executes the complete model with the native PyTorch/cuDNN backend [6] with cuDNN autotuning enabled.
- **Tile skipping** uses DeltaCNN's default execution path for FireFlowNet with Ev-Conv and YOLO [29]. DynConv supplies output masks directly, so tile skipping bypasses only DeltaCNN's mask-propagation step.
- **Gather-scatter** uses DynConv's original CUDA backend for pose [36]. FireFlowNet and YOLO use MMCV's native MaskedConv2d CUDA path [5]; a thin wrapper only supplies the output mask and applies the workload-specific state update.

Measurements. We measure per-frame full-model GPU latency over the complete evaluation datasets. CUDA events bracket each frame's inference call; after workload-specific warm-up we run three complete rounds and average per-frame latency across all frames and rounds. The bracketed interval includes nonblocking activity polling and bucket lookup, mask propagation, worklist construction, all network operators, and output update; it excludes input transfer, offline calibration and autotuning, and one-time state initialization. We report η, T, and T_{eff} at the model level by accumulating required and issued convolution work across all layers within each frame before applying Eq. 3, then averaging across frames. On the Jetson platforms, we integrate board-level `VDD_IN` power over each frame's CUDA-event interval to obtain per-frame board energy, then average across frames. For accuracy, we follow the original workload protocols: AEE for optical flow [8], COCO-style average precision (AP) for detection, averaged over IoU thresholds from 0.50 to 0.95 [29], and PCKh for human pose [36]. Because MOT16 releases ground-truth detections only for its training split, AP on the test sequences uses fixed pseudo-labels from a dense YOLO26x teacher [20].

6.2 End-to-End Performance

Latency. WISEConv consistently converts upstream sparsity into lower full-model latency across all evaluated workloads and platforms. It is the fastest path in all 24 measured workload-platform pairs (Figure 3). Relative to the fastest evaluated competing path in each pair, it achieves a 1.57× geometric-mean speedup; individual-pair speedups range from 1.28× to 1.87×, corresponding to 22.1–46.5% lower latency. These gains persist despite a more than two-order-of-magnitude span in absolute latency, from 0.309 ms for FireFlowNet on RTX 4090 to 94.3 ms for YOLOv8m on Xavier NX.

Unlike existing sparse paths, WISEConv delivers uniform gains across workload regimes. Dense is the fastest evaluated competing path in 15 of the 24 pairs, compared with eight for tile skipping and only one for gather-scatter. FireFlowNet's low active ratio gives tile skipping enough headroom to beat Dense on five of six platforms and reduce its latency by up to 42.0%. The higher active ratios of YOLOv8n and YOLOv8m leave less removable work: tile skipping reaches only near parity with Dense on AGX Orin and is slower in the other ten YOLO pairs. Gather-scatter becomes the fastest competing path only for YOLOv8m on AGX Orin, where the heavier convolution amortizes extraction and writeback. DynConv exposes the other limit: despite having the lowest active ratio, tile skipping beats Dense only on the two Jetson platforms. Sparsity magnitude alone therefore does not determine where an existing sparse path crosses the dense baseline.

WISEConv's lead over the per-pair fastest competing path is narrower on the embedded platforms (geometric mean 1.48× on Jetson vs. 1.62× on discrete GPUs), though it remains fastest in all eight Jetson pairs. The gap narrows because sparse paths themselves are more effective on Jetson: the fastest sparse path beats Dense in six of eight Jetson pairs but only three of 16 discrete-GPU pairs (geometric-mean speedup over Dense: 1.26× vs. 0.80×). Dense convolution is more latency-bound on the narrower Jetson GPUs, so the same active ratio yields a larger absolute latency reduction, making sparse paths more likely to outperform dense.

WISEConv also realizes more of the available sparsity headroom than every competing path. It is the closest path to the ideal proportional-scaling reference in every pair, running at 1.09–4.18× the reference while the fastest evaluated competing path remains at 1.88–6.43×. The remaining gap reflects work that does not shrink with convolution activity, including mask processing, worklist construction, and other non-convolution operators. DynConv lies at the upper end despite its low active ratio because every gated block still runs learned mask-prediction kernels and exposes exceptionally short worklists, as further discussed in §6.3.

Energy. WISEConv's latency gains also reduce per-frame board energy on both Jetson platforms. It is the lowest-energy path for every workload (Table 2), reducing energy by 17.0–39.8% relative to the lowest-energy competing path in each pair and by 28.5–72.6% relative to dense execution.

Accuracy. WISEConv's latency and energy gains do not alter the numerical output of the selected sparsity policies. For FireFlowNet and DynConv, its AEE and PCKh match the other sparse paths at the reported precision. In the YOLOv8 models, small floating-point differences accumulate across frames under temporal execution; even so, WISEConv's pseudo-label AP differs from the other sparse paths by at most 0.5% in relative terms (Figure 4), confirming output agreement across execution backends. Compared with dense execution, the upstream sparsity policies lower AP and PCKh by only 1.6–3.5%, while lowering FireFlowNet's AEE by 0.8%.

Table 2. Per-frame board energy (J) on the Jetson platforms. Parentheses report each sparse path's change relative to Dense.

GPU	System	FireFlowNet	YOLOv8n	YOLOv8m	DynConv
AGX Orin	Dense	0.26	0.48	2.70	1.20
	Tile skip	0.15 (-43.8%)	0.44 (-9.1%)	2.33 (-13.7%)	0.67 (-44.7%)
	Gather	0.29 (+9.5%)	0.70 (+46.1%)	2.42 (-10.4%)	2.37 (+97.0%)
	WISEConv	**0.10** (-61.6%)	**0.32** (-33.9%)	**1.57** (-41.7%)	**0.48** (-60.4%)
Xavier NX	Dense	0.28	0.49	2.58	1.08
	Tile skip	0.13 (-54.5%)	0.42 (-13.8%)	2.27 (-12.2%)	0.54 (-50.4%)
	Gather	0.33 (+17.0%)	0.85 (+75.8%)	3.18 (+23.2%)	2.83 (+160.9%)
	WISEConv	**0.08** (-72.6%)	**0.35** (-28.5%)	**1.78** (-31.1%)	**0.34** (-68.5%)

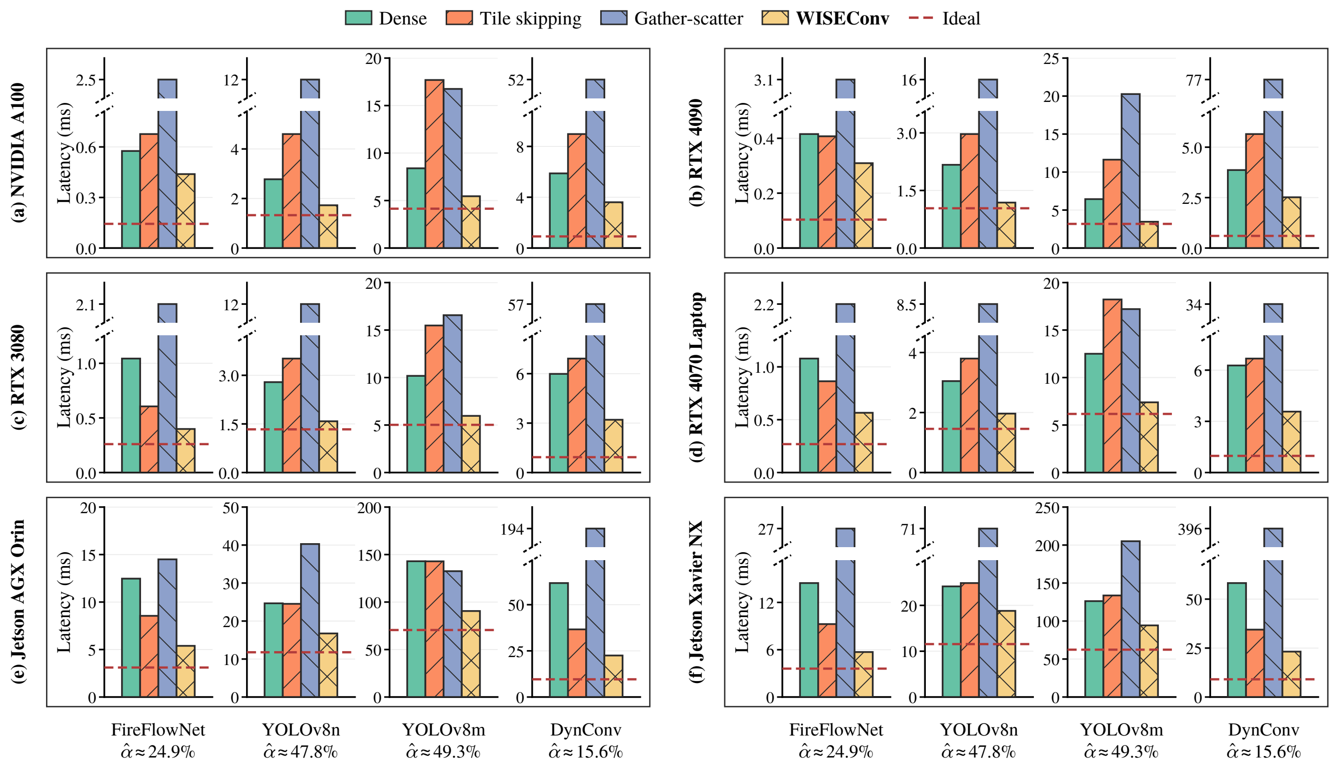


Figure 3. Full-model mean per-frame latency across GPU platforms. Each workload uses an independent y-axis. Labels report the model-level active ratio $\hat{\alpha}$, aggregated as described in §6.1 and averaged across evaluation sequences. Red dashed lines mark ideal proportional scaling, computed by applying each sequence's model-level active ratio to its dense latency before aggregation.

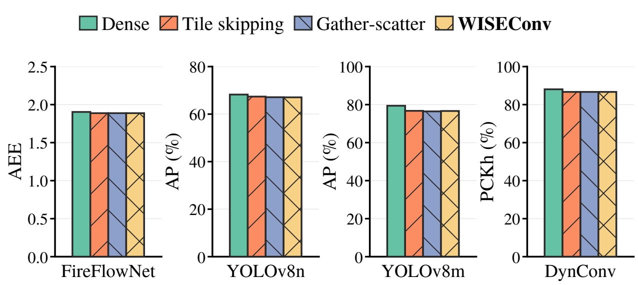


Figure 4. Task accuracy on RTX 3080 under the evaluated upstream sparsity policies, measured by AEE (↓), AP (↑), and PCKh (↑).

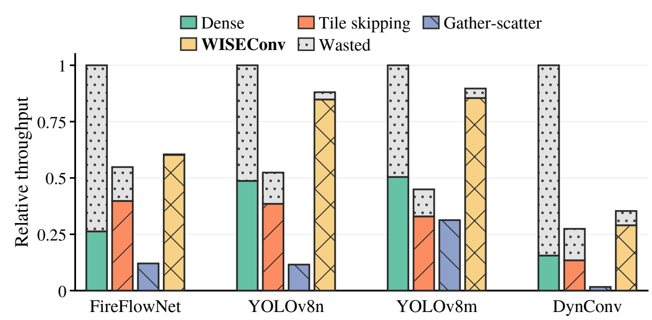


Figure 5. Full-model effective throughput ηT and wasted throughput $(1 - \eta)T$ on RTX 3080, normalized to dense raw throughput.

6.3 Microbenchmarks

Effective throughput. Under the same execution settings as the end-to-end evaluation, we estimate full-model throughput T and effective throughput ηT on RTX 3080 (Figure 5). Table 3 reports WISEConv's η decomposition and worklist-length distribution. WISEConv sustains 35.3–89.7% of dense throughput, the highest among sparse paths in every workload at 1.10–1.99× that of the next-highest sparse path. DeltaCNN's lower raw throughput reflects the impact of routing very sparse tiles through its fused selective path [29]. The two YOLO workloads lie at the upper end because their higher active ratios expose more parallelism and amortize fixed costs.

WISEConv's useful-compute ratio is 98.9%, 96.2%, and 95.2% for FireFlowNet, YOLOv8n, and YOLOv8m. DynConv is lower at 81.7%: its low active ratio and small spatial grids

Table 3. WISEConv η decomposition and worklist-length statistics on RTX 3080. The $n_{\mathcal{W}}$ percentiles are computed from worklist lengths pooled across all layers and frames.

Workload	$\eta_{\text{construct}}$	η_{compute}	η	$n_{\mathcal{W}}$		
				P10	**P50**	**P90**
FireFlowNet	99.9%	99.1%	98.9%	1,303	15,083	33,880
YOLOv8n	98.5%	97.7%	96.2%	77	937	5,937
YOLOv8m	98.7%	96.5%	95.2%	75	840	6,110
DynConv	100.0%	81.7%	81.7%	3	43	389

yield exceptionally short worklists (P10, P50, and P90 lengths of 3, 43, and 389), making final-tile padding proportionally large, and trace-driven adaptation is disabled on this workload (§6.1), so the decomposition cannot adjust per frame to mitigate it. Even under these conditions, WISEConv's useful-compute ratio nearly doubles tile skipping's 48.9%, confirming that the worklist-driven execution model under static decomposition already substantially raises useful-compute ratio. Combining throughput and useful-compute ratio, WISEConv achieves the highest effective throughput in every workload, reaching 1.51–1.86× that of the strongest evaluated competing path.

Table 3 localizes the remaining wasted work: despite cross-operator reuse, $\eta_{\text{construct}}$ remains at 98.5–100.0%. The selected decompositions give η_{compute} values of 99.1%, 97.7%, and 96.5% for FireFlowNet, YOLOv8n, and YOLOv8m; because DynConv has $\eta_{\text{construct}} = 100.0\%$, its lower η comes entirely from compute-stage padding of its short worklists.

Table 4. YOLOv8n latency decomposition on RTX 3080 into worklist construction (Cons.), convolution (Conv.), elementwise operators (Elem., e.g., activation and normalization), and other work, with stage entries in ms and percentages of total latency in parentheses.

	System	Tot.	Cons.	Conv.	Elem.	Other
Low (40.7%)	Dense	2.79	–	2.54(91.2%)	0.14(5.0%)	0.10(3.7%)
	Tile skipping	3.42	–	3.09(90.3%)	0.16(4.6%)	0.17(5.1%)
	Gather	11.81	–	0.73(6.2%)	0.09(0.7%)	11.00(93.1%)
	WISEConv	1.47	0.10(7.1%)	1.09(73.8%)	0.12(8.2%)	0.16(10.9%)
High (56.7%)	Dense	2.79	–	2.54(91.2%)	0.14(5.0%)	0.10(3.7%)
	Tile skipping	3.59	–	3.25(90.4%)	0.17(4.7%)	0.18(4.9%)
	Gather	11.82	–	0.83(7.0%)	0.09(0.8%)	10.90(92.2%)
	WISEConv	1.69	0.11(6.2%)	1.29(76.3%)	0.13(7.7%)	0.17(9.8%)

Notes. Low and High median-split the evaluated MOT16 frames by active ratio; labels report bin means. Each stage time multiplies its per-frame NSYS fraction by the corresponding end-to-end CUDA-event latency.

Latency cost decomposition. We decompose YOLOv8n latency on RTX 3080 by execution stage (Table 4), where frames from MOT16 [25] are classified into bins of low or high active ratios. WISEConv's construction time remains at 0.105–0.106 ms while convolution grows from 1.09 to 1.29 ms across the two bins; construction's latency share consequently falls from 7.1% to 6.2%. Gather-scatter shows why exact selection alone is insufficient: it has the lowest convolution time at 0.73–0.83 ms, but 10.9–11.0 ms of auxiliary work outside convolution arithmetic (92.2–93.1% of total latency) is reported as *Other*, compared with 0.10 ms for Dense. For gather-scatter, this category captures its auxiliary path (active-coordinate extraction, receptive-field materialization, GEMM setup, and scatter/writeback); for all paths it also includes inter-kernel idle gaps from launch and dependency serialization. In contrast, WISEConv adds only 0.06–0.07 ms of non-construction work over Dense. Direct worklist consumption thus avoids this auxiliary path while keeping convolution dominant.

6.4 Ablation Study and Sensitivity Analysis

Table 5. Component ablation for YOLOv8n on RTX 3080, using the same dataset and stage-attribution setting as Table 4. Each variant disables one mechanism while keeping the rest identical to Ours.

Variant	η	Cons.	Conv.	Total
Atomic append	95.65%	0.182	1.184	1.684
Global order	95.60%	0.246	1.163	1.719
No reuse	97.02%	0.205	1.168	1.680
Static level	95.61%	0.105	1.231	1.642
Ideal level	95.79%	0.105	1.141	1.543
Ours	95.57%	0.105	1.174	1.583

Notes. Atomic append: per-coordinate atomic reservation, losing warp compaction and tile-local segments. *Global order*: stable prefix-scan compaction over the full output grid, replacing tile-local construction. *No reuse*: every Conv2d and activation reconstructs its worklist, disabling cross-operator reuse. *Static level*: per-layer fixed decomposition calibrated offline, disabling adaptive level selection. *Ideal level*: oracle that observes the current worklist length and selects the optimal level per layer per frame (GPU lower bound, not deployable).

Each row in Table 5 disables one mechanism. Per-coordinate atomics (Atomic append) raise construction to 0.182 ms through finer serialization; convolution also rises to 1.184 ms because the resulting worklist loses tile-local contiguity. A full-grid prefix-scan (Global order) raises construction to 0.246 ms; convolution drops marginally to 1.163 ms, but the 0.141 ms construction overhead far exceeds the 0.011 ms compute benefit, validating tile-local segments as a sufficient approximation. Disabling cross-operator reuse (No reuse) nearly doubles construction to 0.205 ms while convolution is unchanged, confirming that reuse eliminates redundant construction cost. Fixing per-layer decomposition offline (Static level) raises convolution by 4.9%: frames whose worklist length deviates from the calibration point receive a mismatched configuration. The Ideal level oracle reduces total latency by only 2.5% below Ours, bounding the cost of lagged observation and showing that a completed, lagged activity observation is sufficient for level selection in this workload.

Table 6. YOLOv8n latency decomposition under stream-level batching on RTX 3080. Each sequence is split into disjoint temporal streams; each batch combines one current frame per stream and advances their states independently. Stage definitions, units, and cell format follow Table 4.

	System	Tot.	Cons.	Conv.	Elem.	Other
Batch size=1	Dense	2.79	–	2.54(91.2%)	0.14(5.0%)	0.10(3.7%)
	Tile skipping	3.47	–	3.13(90.4%)	0.16(4.6%)	0.17(5.0%)
	Gather	11.33	–	0.75(6.6%)	0.09(0.8%)	10.50(92.7%)
	WISEConv	1.56	0.10(6.7%)	1.17(75.0%)	0.12(7.9%)	0.16(10.4%)
Batch size=4	Dense	1.83	–	1.61(88.0%)	0.15(8.0%)	0.07(4.0%)
	Tile skipping	1.66	–	1.41(84.7%)	0.12(7.4%)	0.13(7.8%)
	Gather	6.41	–	0.60(9.4%)	0.09(1.4%)	5.71(89.2%)
	WISEConv	1.05	0.03(3.1%)	0.80(76.5%)	0.09(8.8%)	0.12(11.6%)
Batch size=8	Dense	1.68	–	1.47(87.3%)	0.15(8.9%)	0.06(3.9%)
	Tile skipping	1.41	–	1.17(83.0%)	0.12(8.4%)	0.12(8.6%)
	Gather	3.80	–	0.56(14.7%)	0.09(2.3%)	3.15(83.0%)
	WISEConv	0.94	0.02(2.2%)	0.72(76.3%)	0.09(9.3%)	0.11(12.2%)

Stream-level batching. Table 6 shows how stream-level batching changes per-frame costs. At batch size 8, gather-scatter still spends 83.0% of its latency in *Other*, whereas tile skipping becomes the fastest competing path at batch sizes 4 and 8. Its launch grid is determined statically from the input shape, so increasing the batch dimension adds independent tiles from multiple temporal streams, consistent with DeltaCNN's observation for high-end GPUs [29]. The effect is concentrated in convolution: from batch size 1 to 8, tile skipping's per-frame convolution cost falls 2.68×, compared with 1.63× for WISEConv. WISEConv remains fastest, but its lead narrows from 1.79× to 1.49×. These results, in turn, show why adaptive decomposition matters at batch size 1: WISEConv adapts its work decomposition to each short worklist instead of relying on a larger batch to expand the launch grid.

Table 7. DynConv pose latency decomposition on RTX 3080 under FP32 (SIMT) and FP16 (tensor core) execution. Same stage definitions and timing as Table 4; entries are ms with percentages of total latency in parentheses.

Type	System	Tot.	Cons.	Conv.	Elem.	Other
FP32	Dense	5.99	–	5.25(87.7%)	0.09(1.5%)	0.65(10.8%)
	WISEConv	3.20	0.09(2.9%)	2.18(68.2%)	0.07(2.1%)	0.86(26.7%)
FP16	Dense	3.37	–	2.22(66.0%)	0.05(1.5%)	1.10(32.5%)
	WISEConv	2.57	0.08(3.0%)	1.38(53.4%)	0.06(2.5%)	1.06(41.1%)

Tensor core sensitivity. Table 7 decomposes DynConv pose latency under FP16 (tensor core) execution alongside the FP32 (SIMT) baseline. WISEConv still reduces Dense FP16 latency from 3.37 to 2.57 ms (1.31×), though the gain is narrower than the 1.87× in FP32. Two factors outside the worklist-driven convolution account for this. First, tensor cores compress the arithmetic phase of each kernel but not its fixed pipeline cost (startup, drain, scheduling); this fixed cost therefore becomes more dominant, and removing the same proportion of arithmetic yields less wall-clock saving (absolute convolution saving falls from 3.07 in FP32 to 0.84 ms in FP16). Second, shorter kernels expose more inter-kernel idle time, causing *Other* to grow from 0.65 to 1.10 ms for Dense and from 0.86 to 1.06 ms for WISEConv (33–41% of FP16 latency). Construction remains 0.08 ms (3.0%), unchanged from FP32, so the narrower gain is not caused by worklist overhead. FP16 PCKh (88.07% Dense, 86.68% WISEConv) matches FP32 within 0.01 percentage points; the datatype introduces no evident accuracy difference.

6.5 Discussion

WISEConv consumes rather than generates masks, so its gains are conditional on the upstream policy and its activity distribution; it guarantees coverage of requested outputs but does not improve mask quality. Across our workloads, the evaluation shows that WISEConv most effectively converts the supplied mask sparsity into end-to-end latency reduction among the evaluated sparse paths. The adaptive decomposition is deployment-specific and must be recalibrated for a new model, device, or mask workload, although prediction error affects performance rather than correctness.

The full workload comparison uses FP32 (SIMT): the evaluated masked-convolution baselines [5, 29, 36] provide only FP32 execution paths, so FP32 isolates execution-model differences from arithmetic-precision effects. Extending the worklist compute stage to tensor cores also raises an open reduction problem: split-K partitions must atomically accumulate FP32 partial sums and convert them back to FP16, and for short worklists this overhead can outweigh the FP16 arithmetic savings. The targeted DynConv study confirms that WISEConv retains an end-to-end gain under tensor cores; a dedicated tensor core reduction design for short worklists is a natural next step.

7 Conclusion

We presented WISEConv, a masked-convolution runtime that replaces the dense tiled pipeline's contiguous coordinate stream with an active-output worklist, computing only positions an upstream mask marks active while retaining the dense tensor layout and the regular per-position reduction. Construction completes in a single mask-space pass and is reused across compatible operators; consumption preserves tile-local ordering, bounds work with a device-resident worklist length, and adapts its decomposition to input activity without host synchronization. On the evaluated workload and device, disabling any one of these mechanisms measurably raises latency through either excessive construction cost or underloaded consumption. Together they secure both useful-compute ratio and throughput, converting upstream mask sparsity into end-to-end latency and energy reduction. With WISEConv, spatial masks consistently reduce per-frame GPU latency across the evaluated traces, making activity-driven scheduling practical on latency-constrained deployed perception platforms.

References

[1] 2026. CUTLASS: CUDA Templates and Python DSLs for High-Performance Linear Algebra. https://github.com/NVIDIA/cutlass

[2] Mykhaylo Andriluka, Leonid Pishchulin, Peter Gehler, and Bernt Schiele. 2014. 2D Human Pose Estimation: New Benchmark and State of the Art Analysis. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 3686–3693.

[3] Rik J. Bouwmeester, Federico Paredes-Vallés, and Guido C. H. E. de Croon. 2023. NanoFlowNet: Real-time Dense Optical Flow on a Nano Quadcopter. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*. 1996–2003.

[4] Lukas Cavigelli, Philippe Degen, and Luca Benini. 2017. CBinfer: Change-Based Inference for Convolutional Neural Networks on Video Data. In *Proc. Int. Conf. Distrib. Smart Cameras (ICDSC)*. 1–8.

[5] Kai Chen, Jiaqi Wang, Jiangmiao Pang, Yuhang Cao, Yu Xiong, Xiaoxiao Li, Shuyang Sun, Wansen Feng, Ziwei Liu, Jiarui Xu, Zheng Zhang, Dazhi Cheng, Chenchen Zhu, Tianheng Cheng, Qijie Zhao, Buyu Li, Xin Lu, Rui Zhu, Yue Wu, Jifeng Dai, Jingdong Wang, Jianping Shi, Wanli Ouyang, Chen Change Loy, and Dahua Lin. 2019. MMDetection: Open MMLab Detection Toolbox and Benchmark. *CoRR* abs/1906.07155 (2019).

[6] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. 2014. cuDNN: Efficient Primitives for Deep Learning. *CoRR* abs/1410.0759 (2014).

[7] Christopher B. Choy, JunYoung Gwak, and Silvio Savarese. 2019. 4D Spatio-Temporal ConvNets: Minkowski Convolutional Neural Networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

[8] Sankeerth Durvasula, Yushi Guan, and Nandita Vijaykumar. 2023. Ev-Conv: Fast CNN Inference on Event Camera Inputs for High-Speed Robot Perception. *IEEE Robotics Autom. Lett.* 8, 6 (2023), 3174–3181.

[9] Davide Falanga, Philipp Foehn, Peng Lu, and Davide Scaramuzza. 2018. PAMPC: Perception-Aware Model Predictive Control for Quadrotors. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS 2018, Madrid, Spain, October 1-5, 2018*. IEEE, 1–8. doi:10.1109/IROS.2018.8593739

[10] Davide Falanga, Suseong Kim, and Davide Scaramuzza. 2019. How Fast Is Too Fast? The Role of Perception Latency in High-Speed Sense and Avoid. *IEEE Robotics Autom. Lett.* 4, 2 (2019), 1884–1891. doi:10.1109/LRA.2019.2898117

[11] Davide Falanga, Kevin Kleber, and Davide Scaramuzza. 2020. Dynamic obstacle avoidance for quadrotors with event cameras. *Sci. Robotics* 5, 40 (2020), 9712. doi:10.1126/SCIROBOTICS.AAZ9712

[12] Ruibo Fan, Wei Wang, and Xiaowen Chu. 2024. DTC-SpMM: Bridging the Gap in Accelerating General Sparse Matrix Multiplication with Tensor Cores. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024- 1 May 2024*, Rajiv Gupta, Nael B. Abu-Ghazaleh, Madan Musuvathi, and Dan Tsafrir (Eds.). ACM, 253–267. doi:10.1145/3620666.3651378

[13] Michael Figurnov, Maxwell D. Collins, Yukun Zhu, Li Zhang, Jonathan Huang, Dmitry P. Vetrov, and Ruslan Salakhutdinov. 2017. Spatially Adaptive Computation Time for Residual Networks. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 1790–1799.

[14] Trevor Gale, Matei Zaharia, Cliff Young, and Erich Elsen. 2020. Sparse GPU Kernels for Deep Learning. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*.

[15] Yizhao Gao, Baoheng Zhang, Xiaojuan Qi, and Hayden Kwok-Hay So. 2023. DPACS: Hardware Accelerated Dynamic Neural Network Pruning through Algorithm-Architecture Co-design. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2023, Vancouver, BC, Canada, March 25-29, 2023*, Tor M. Aamodt, Natalie D. Enright Jerger, and Michael M. Swift (Eds.). ACM, 237–251. doi:10.1145/3575693.3575728

[16] Benjamin Graham, Martin Engelcke, and Laurens van der Maaten. 2018. 3D Semantic Segmentation with Submanifold Sparse Convolutional Networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

[17] Amirhossein Habibian, Davide Abati, Taco S. Cohen, and Babak Ehteshami Bejnordi. 2021. Skip-Convolutions for Efficient Video Processing. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 2695–2704.

[18] Drew Hanover, Antonio Loquercio, Leonard Bauersfeld, Angel Romero, Robert Penicka, Yunlong Song, Giovanni Cioffi, Elia Kaufmann, and Davide Scaramuzza. 2024. Autonomous Drone Racing: A Survey. *IEEE Trans. Robotics* 40 (2024), 3044–3067. doi:10.1109/TRO.2024.3400838

[19] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. 2023. Ultralytics YOLOv8. https://github.com/ultralytics/ultralytics

[20] Glenn Jocher, Jing Qiu, Mengyu Liu, Shuai Lyu, Fatih Cagatay Akyon, and Muhammet Esat Kalfaoglu. 2026. Ultralytics YOLO26: Unified Real-Time End-to-End Vision Models. *CoRR* abs/2606.03748 (2026).

[21] Fredrik Kjolstad, Shoaib Kamil, Stephen Chou, David Lugato, and Saman P. Amarasinghe. 2017. The Tensor Algebra Compiler. *Proc. ACM Program. Lang.* 1, OOPSLA (2017).

[22] Fanrong Li, Gang Li, Xiangyu He, and Jian Cheng. 2021. Dynamic Dual Gating Neural Networks. In *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*. 5310–5319.

[23] Renyuan Liu, Yuyang Leng, Shilei Tian, Shaohan Hu, Chun-Fu (Richard) Chen, and Shuochao Yao. 2024. DynaSpa: Exploiting Spatial Sparsity for Efficient Dynamic DNN Inference on Devices. In *Proceedings of the 22nd ACM Conference on Embedded Networked Sensor Systems, SenSys 2024, Hangzhou, China, November 4-7, 2024*. ACM, 422–435. doi:10.1145/3666025.3699348

[24] Nico Messikommer, Daniel Gehrig, Antonio Loquercio, and Davide Scaramuzza. 2020. Event-Based Asynchronous Sparse Convolutional Networks. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*. 415–431.

[25] Anton Milan, Laura Leal-Taixé, Ian D. Reid, Stefan Roth, and Konrad Schindler. 2016. MOT16: A Benchmark for Multi-Object Tracking. *CoRR* abs/1603.00831 (2016).

[26] Asit K. Mishra, Jorge Albericio Latorre, Jeff Pool, Darko Stosic, Dusan Stosic, Ganesh Venkatesh, Chong Yu, and Paulius Micikevicius. 2021. Accelerating Sparse Deep Neural Networks. *CoRR* abs/2104.08378 (2021).

[27] Federico Paredes-Vallés and Guido C. H. E. de Croon. 2021. Back to Event Basics: Self-Supervised Learning of Image Reconstruction for Event Cameras via Photometric Constancy. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 3445–3454.

[28] Mathias Parger, Chengcheng Tang, Thomas Neff, Christopher D. Twigg, Cem Keskin, Robert Wang, and Markus Steinberger. 2023. MotionDeltaCNN: Sparse CNN Inference of Frame Differences in Moving Camera Videos with Spherical Buffers and Padded Convolutions. In *IEEE/CVF International Conference on Computer Vision, ICCV 2023, Paris, France, October 1-6, 2023*. 17246–17255.

[29] Mathias Parger, Chengcheng Tang, Christopher D. Twigg, Cem Keskin, Robert Wang, and Markus Steinberger. 2022. DeltaCNN: End-to-End CNN Inference of Sparse Frame Differences in Videos. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 12487–12496.

[30] Hongwu Peng, Xi Xie, Kaustubh Shivdikar, Md Amit Hasan, Jiahui Zhao, Shaoyi Huang, Omer Khan, David R. Kaeli, and Caiwen Ding.

2024. MaxK-GNN: Extremely Fast GPU Kernel Design for Accelerating Graph Neural Networks Training. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024- 1 May 2024*, Rajiv Gupta, Nael B. Abu-Ghazaleh, Madan Musuvathi, and Dan Tsafrir (Eds.). ACM, 683–698. doi:10.1145/3620665.3640426

[31] Mengye Ren, Andrei Pokrovsky, Bin Yang, and Raquel Urtasun. 2018. SBNet: Sparse Blocks Network for Fast Inference. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 8711–8720.

[32] Spconv Contributors. 2022. Spconv: Spatially Sparse Convolution Library. https://github.com/traveller59/spconv.

[33] Martin Sundermeyer, Arsalan Mousavian, Rudolph Triebel, and Dieter Fox. 2021. Contact-GraspNet: Efficient 6-DoF Grasp Generation in Cluttered Scenes. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*. 13438–13444.

[34] Haotian Tang, Zhijian Liu, Xiuyu Li, Yujun Lin, and Song Han. 2022. TorchSparse: Efficient Point Cloud Inference Engine. In *Proceedings of Machine Learning and Systems (MLSys)*.

[35] Haotian Tang, Shang Yang, Zhijian Liu, Ke Hong, Zhongming Yu, Xiuyu Li, Guohao Dai, Yu Wang, and Song Han. 2023. TorchSparse++: Efficient Training and Inference Framework for Sparse Convolution on GPUs. In *Proc. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*. 225–239.

[36] Thomas Verelst and Tinne Tuytelaars. 2020. Dynamic Convolutions: Exploiting Spatial Sparsity for Faster Inference. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 2320–2329.

[37] Chen Wang, Danfei Xu, Yuke Zhu, Roberto Martin Martin, Cewu Lu, Li Fei-Fei, and Silvio Savarese. 2019. DenseFusion: 6D Object Pose Estimation by Iterative Dense Fusion. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 3343–3352.

[38] Kunyun Wang, Shuo Yang, Jieru Zhao, Wenchao Ding, Quan Chen, Jingwen Leng, and Minyi Guo. 2025. SparseTem: Boosting the Efficiency of CNN-Based Video Encoders by Exploiting Temporal Continuity. In *Advanced Parallel Processing Technologies - 16th International Symposium, APPT 2025, Athens, Greece, July 13-16, 2025, Proceedings*. 246–256.

[39] Diana Wofk, Fangchang Ma, Tien-Ju Yang, Sertac Karaman, and Vivienne Sze. 2019. FastDepth: Fast Monocular Depth Estimation on Embedded Systems. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*. 6101–6108.

[40] Zhenda Xie, Zheng Zhang, Xizhou Zhu, Gao Huang, and Stephen Lin. 2020. Spatially Adaptive Inference with Stochastic Feature Sampling and Interpolation. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*. 531–548.

[41] Jiacheng Yang, Christina Giannoula, Jun Wu, Mostafa Elhoushi, James Gleeson, and Gennady Pekhimenko. 2024. Minuet: Accelerating 3D Sparse Convolutions on GPUs. In *Proceedings of the Nineteenth European Conference on Computer Systems, EuroSys 2024, Athens, Greece, April 22-25, 2024*. 786–802.

[42] Zihao Ye, Ruihang Lai, Junru Shao, Tianqi Chen, and Luis Ceze. 2023. SparseTIR: Composable Abstractions for Sparse Compilation in Deep Learning. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.

[43] Changqian Yu, Jingbo Wang, Chao Peng, Changxin Gao, Gang Yu, and Nong Sang. 2018. BiSeNet: Bilateral Segmentation Network for Real-Time Semantic Segmentation. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*. 334–349.

[44] Alex Zihao Zhu, Dinesh Thakur, Tolga Özaslan, Bernd Pfrommer, Vijay Kumar, and Kostas Daniilidis. 2018. The Multi Vehicle Stereo Event Camera Dataset: An Event Camera Dataset for 3D Perception. *CoRR* abs/1801.10202 (2018).